

[Jogo da Velha]

Interface Visual com KivyMD

Integrantes do Grupo

- Vinicius Brito — [2840482421023]

O Jogo

Regras, objetivo e o que foi implementado na Fase 1

Sobre o Jogo

Objetivo: [O jogo é disputado por dois jogadores que se alternam marcando células com seus símbolos (X ou O). O objetivo é alinhar três símbolos iguais, seja na horizontal, vertical ou diagonal antes do adversário. Se todas as células forem preenchidas sem que nenhum jogador complete uma linha, a rodada termina em empate]

Número de jogadores: [2]

Tipo de tabuleiro: [grade 3x3]

Regras principais:

- Cada jogador deve esperar sua vez para realizar sua jogada
- O jogador pode selecionar apenas uma célula por vez
- Não é possível selecionar células já selecionadas

O que fizemos na Fase 1

Classes implementadas:

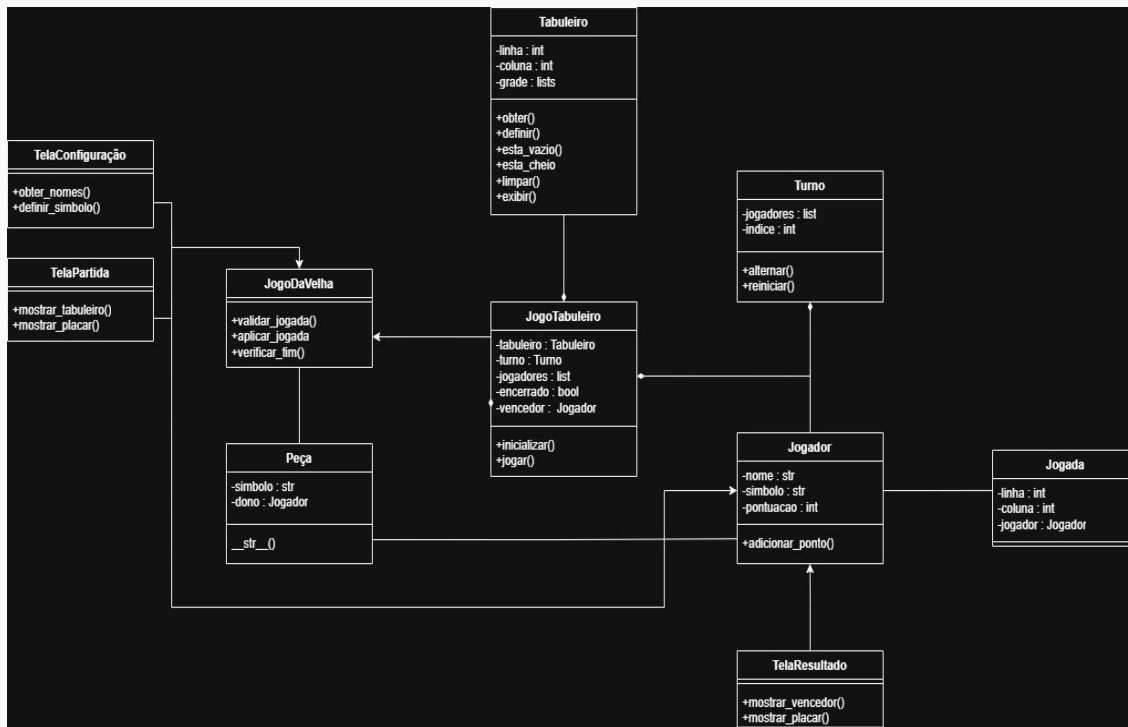
- [jogada] — representa uma jogada feita no tabuleiro
- [jogador] — armazena dados e pontuação do jogador
- [peca] — representa o símbolo colocado na célula
- [jogo] — controla o fluxo e regras da partida
- [tabuleiro] — gerencia o estado da grade 3x3

Lógica central:

[Ao executar o arquivo, o jogo solicitava o nome e símbolo de cada jogador. Com a partida iniciada, os jogadores se alternavam inserindo a linha e coluna da célula desejada. Ao término, era exibido o vencedor e perguntado se desejavam jogar novamente]

Arquitetura OO

Diagrama UML das principais classes e decisões de herança/polimorfismo



Conceitos OO Aplicados



Encapsulamento

Atributos privados em [Tabuleiro], [Jogada], [Peca] e [Turno]. Acesso via getters/setters.



Herança

[JogoDaVelha] herda de [JogoTabuleiro] pois compartilha a estrutura base de um jogo de tabuleiro (tabuleiro, turno e jogadores), especializando apenas regras específicas.



Polimorfismo

Método [validar_jogada()] se comporta diferente em [JogoDaVelha] e [JogoTabuleiro].



Interface / ABC

[JogoTabuleiro] define o contrato para [JogoDaVelha].

Decisões de Design

Por que separamos lógica de UI desse jeito? Quais padrões foram aplicados?

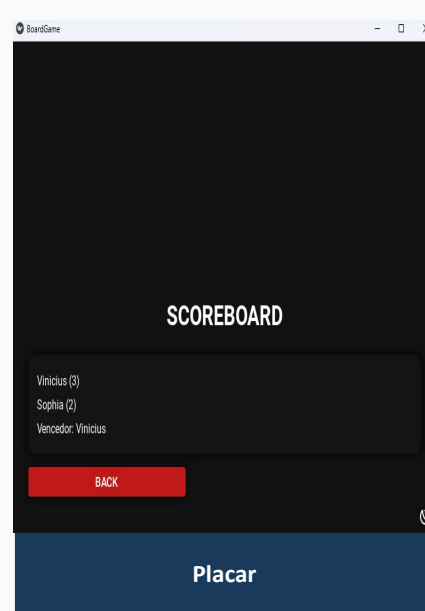
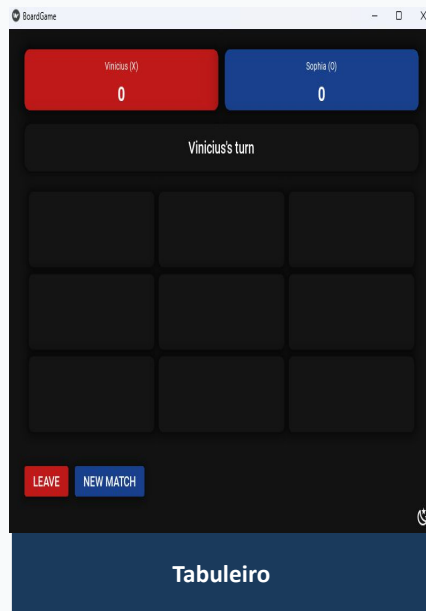
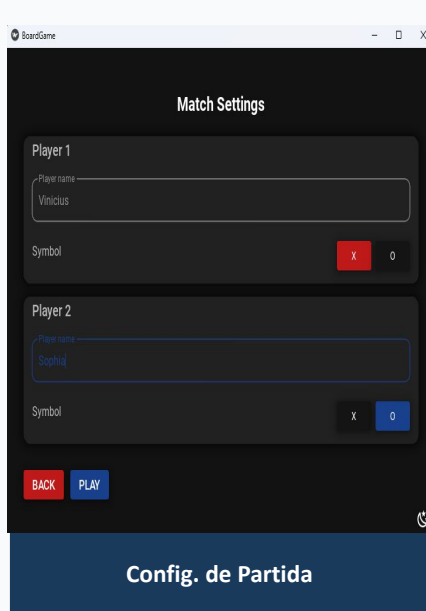
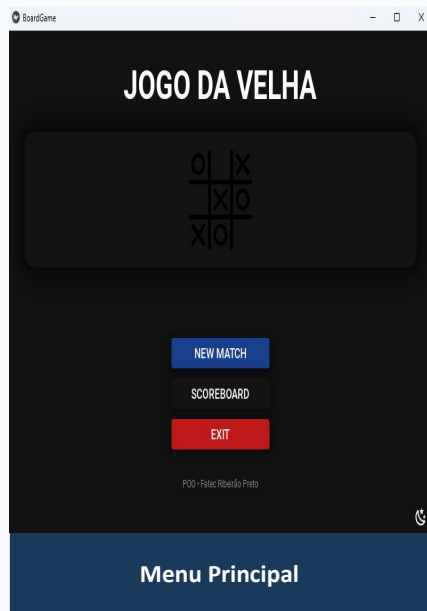


Nossas decisões de design e por que as tomamos

1. [Decisão 1] — Usei o padrão Observer para notificar a UI de mudanças no estado do jogo porque as telas não precisam verificar o estado a cada frame, o GameController dispara o callback `on_board_update` automaticamente após cada jogada, mantendo Model e View desacoplados.
2. [Decisão 2] — As classes do model não importam nada do Kivy, garantindo que possam ser testadas isoladamente porque toda a lógica de jogo foi desenvolvida na Fase 1 sem dependência de interface gráfica, permitindo reutilizá-la integralmente na Fase 2.
3. [Decisão 3] — Escolhi construir a UI via Python em vez de arquivos `.kv` para o tabuleiro porque facilita a criação dinâmica das células e a atualização programática dos widgets em resposta aos eventos do jogo.

Telas Implementadas

Capturas de tela de cada tela com breve legenda



💡 Substitua os placeholders acima por capturas de tela reais do seu aplicativo (Insert → Pictures)



Demonstração ao Vivo

[Apresentar o jogo rodando — nova partida → jogadas → resultado]

Roteiro sugerido para a demo: Iniciar partida → Inserir nomes → Realizar 2-3 jogadas → Mostrar detecção de fim de jogo

Desafios & Aprendizados

O que foi mais difícil? Como resolvemos? O que faríamos diferente?



Maior Desafio

[A maior dificuldade da Fase 2 foi a integração das telas utilizando o KivyMD, framework que eu ainda não havia utilizado. Enquanto a lógica Orientada a Objetos da Fase 1 já estava consolidada, a curva de aprendizado do Kivy trouxe desafios específicos como entender o ciclo de vida das telas, a hierarquia de widgets e principalmente a comunicação entre as telas e a lógica do jogo através do GameController]



Como Resolvemos

[Esses desafios foram superados através da pesquisa contínua na documentação do Kivy e de testes incrementais, implementando uma tela por vez e validando seu funcionamento antes de avançar para a próxima. A comunicação entre View e Model foi resolvida com a criação do GameController, que centraliza toda a lógica. Erros de integração, como referências incorretas entre métodos e atributos, foram corrigidos por meio de revisões do código com auxílio da IA, comparando o comportamento esperado com o resultado obtido a cada execução]



Faríamos Diferente

[Em uma futura iteração, eu adotaria testes automatizados para o GameController antes de conectá-lo as telas, validando a lógica de intermediação isoladamente e reduzindo o tempo gasto depurando erros que só apareciam em tempo de execução. Por fim, estudaria a documentação do Kivy de forma mais estruturada antes de começar a implementação, em vez de aprender o framework de forma reativa, conforme os problemas apareciam]

Conclusão

O que foi entregue, extras implementados e próximos passos



O que entregamos

- [Configuração de jogadores]
- [Sistema de partida MD5]
- [4] telas
- [MVC]
- Repositório público no GitHub
- [Modo claro/escuro]
- [Transições entre telas]



Próximos Passos

- [Oponente IA]
- [Correção de bugs]
- [Scoreboard permanente]
- [Modo online]

Obrigado!

[Jogo da Velha] • Grupo: [Vinicius Brito]

 Repositório: github.com/viniciusbr1to/poo-board-game-python

Perguntas?